

Semantic Transcript Segmentation (Podcast Domain) 工程规划书

1. 项目背景与工程目标

在实际业务场景中，长篇对话式播客（如 Lex Fridman, Huberman Lab 等）缺乏天然的章节结构。这导致下游的 RAG (检索增强生成) 和 Agent 系统在处理时，往往采用暴力的“固定长度/固定段落”切分，严重破坏了上下文逻辑，造成信息跨段丢失和语义截断。

本项目旨在研发一个基于轻量级模型的**语义边界探测系统 (Semantic Boundary Detection)**，通过识别文本的深层“逻辑跃迁”，实现播客等单人/长文本 Transcript 的智能切分。

核心工程目标：

1. 跑通从数据抓取、大模型蒸馏（自动标注）、数据工程到模型微调、部署的完整 ML Pipeline。
2. 在“深度科技播客”垂直领域，核心指标（语义边界命中率/下游检索召回率）击败现有的通用 Embedding 阈值切分基线。
3. （可选）封装为可复用的 Python Package 进行开源。

2. 技术栈架构

- **数据采集与处理：** `youtube-transcript-api`, `pandas`, `scikit-learn`
- **基线测试与分析：** `sentence-transformers` (例如 `bge-large-en`), `nltk` (`TextTiling`), `LlamaIndex` (`SemanticSplitter`)
- **Teacher 自动标注：** OpenAI API (`gpt-4o` 验证, `gpt-4o-mini` + Batch API 铺量)
- **核心模型训练：** `PyTorch`, `HuggingFace Transformers`, `ModernBERT-base` (适合长文本、高吞吐)
- **实验与指标追踪：** `Weights & Biases` (WandB)

3. 核心执行路线图 (Milestones)

Phase 0: 摸底测试与错题本 (Baseline Benchmark & Error Analysis)

- **目标：** 不急于训练，先确立 Baseline，用真实数据暴露通用方案在垂直场景中的缺陷。
- **执行动作：**
 1. **Golden Test Set 构造：** 抓取 3-5 篇带有高质量 YouTube Chapters (时间戳目录) 的科技类播客 Transcript。将时间戳转化为 Ground Truth `[0, 0, 1, 0...]` 边界标签。
 2. **运行 Baseline：** * 运行 `LlamaIndex` 的 `Semantic Splitter` (基于通用 Embedding 的余弦相似度)。
 - 运行传统的 `TextTiling` 算法。
 3. **Error Analysis (关键步)：** 重点提取假阳性 (False Positives - 误切) 和假阴性 (False Negatives - 漏切) 的样本。

气词敏感等)。

Phase 1: Teacher LLM 标注流水线 MVP (Pipeline MVP)

- **目标：** 打造稳定、高质量的合成数据生产线。
- **执行动作：**
 1. **Prompt Engineering：** 编写针对 GPT 模型的 System Prompt，明确“实质性话题转移”与“同话题内逻辑递进”的区别。
 2. **局部验证：** 在 Phase 0 的 5 篇文本上，使用 gpt-4o 和 gpt-4o-mini 同时进行自动标注。
 3. **对齐检验：** 抽样 50-100 个边界，人工检验 LLM 的判断是否符合人类的高级逻辑直觉。如果 gpt-4o-mini 表现达标，确定将其作为 Phase 2 的主力 Teacher。
- **产出物：** 经过验证的 Prompt 模板与自动化调用脚本。

Phase 2: 大规模数据集构建与工程处理 (Data Engineering)

- **目标：** 产出格式规范、分布相对健康的训练准备数据集 (Ready-to-train Dataset)。
- **执行动作：**
 1. **批量抓取：** 获取 50-100 篇同垂直领域的播客文本。
 2. **异步标注：** 使用 OpenAI Batch API 跑完全量数据，获取约 5k - 20k 的候选边界样本。
 3. **格式转换：** 将结果解析为监督学习格式：(Context_Prev, Segment_A, Segment_B, Context_Next) -> Label (0 or 1)。

Phase 3: 核心模型微调与策略选择 (Model Fine-tuning & Routing)

- **目标：** 解决长文本分类中典型的**类别极度不平衡 (Class Imbalance)** 与 **注意力稀释 (Attention Dilution)** 问题。
- **执行策略 (两阶段漏斗与决策树)：**
 - **第一步：粗排过滤 (Stage 1 Recall)**
 - 使用现成的开源轻量级模型 (如 bge-small) 计算所有相邻段落的相似度。
 - 设定宽松阈值 (如 Cosine Similarity < 0.8)，将极度连贯的相邻段落直接抛弃。
 - 评估粗排后的正负样本比例 (Label 1 vs Label 0)。
 - **第二步：根据漏斗比例决定精排方案 (Stage 2 Precision Routing)**
 - 🟡 **方案 A: 目标标记二分类 (如果正负比例拉回至 1:2 ~ 1:5)**
 - **输入构造：** 引入 <tgt_a> 和 <tgt_b> 特殊 Token。输入格式：[宏观上文] <tgt_a> 段落 A </tgt_a> [SEP] <tgt_b> 段落 B </tgt_b> [微观下文]。
 - **网络结构：** ModernBERT，提取两个特殊 Token 的 Hidden States 拼接后做 Linear 映射。
 - **损失函数：** 使用 Focal Loss 进一步抑制容易分类的负样本权重。
 - 🟡 **方案 B: 表征学习/对比学习 (如果正负比例依然 > 1:10)**
 - **放弃二分类头，将模型转为专属 Embedding 提取器。**

挖掘“困难负样本 (Hard Negatives)”。

- **网络结构 & 损失：**使用 ModernBERT 提取特征，采用 Triplet Loss 或 InfoNCE Loss ，强行拉开逻辑转折处的向量距离。
- **产出物：**收敛的 ModernBERT Checkpoints，以及 WandB 上的训练指标记录。

Phase 4: 分段系统集成 (Inference Pipeline)

- **目标：**实现从生肉 Transcript 到完美 Chunk 的端到端工具。
- **执行动作：**
 1. 将预测结果 (Probabilities / 向量距离) 结合文本实际长度 (字数/Token数)。
 2. 实现长度惩罚逻辑 (例如: Min_chunk=300 tokens, Max_chunk=1500 tokens)。
 3. 使用局部滑动窗口寻优或动态规划 (Dynamic Programming) 算法，在长度约束内选择概率最高/距离最大的点作为最终切分刀。
- **产出物：**核心的 Segmenter Python Class。

Phase 5: 下游任务评估 (Downstream Task Evaluation)

- **目标：**从业务价值角度，证明语义分段的优越性。
- **执行动作 (二选一或组合)：**
 - **Task A: RAG 事实检索 (Information Retrieval)** * 对特定长播客构建 30-50 个细节 QA 问答对。
 - 对比 通用 Embedding 切分 与 Our Semantic Segmenter 切分 构建的 Vector DB，在 Top-3 召回率上的差异。
 - **Task B: 结构化抽取 (Information Extraction)**
 - 设计特定的 Agent 任务 (如: 提取播客中提到的所有书籍名称及讲者评价)。
 - 评估在何种 Chunking 方案下，LLM 的幻觉更少、信息遗漏率更低。
- **产出物：**完整的对比实验结果与图表。

4. 成功标准与交付物 (Deliverables)

1. **Pipeline 资产：**包含数据爬取、GPT 标注、模型微调、评测的全部脚本库。
2. **数据集：**一个垂直领域 (Podcast Transcript) 的高质量语义边界数据集。
3. **模型资产：**一个针对口语化长文本优化过的轻量级 ModernBERT-Podcast-Segmenter 模型。
4. **项目报告：**一份详实的 Markdown/PDF 报告，清晰阐述从 Baseline 失效到自研模型成功的完整技术逻辑，作为极为硬核的个人简历背书。